THE DEEP DIVE

Mastering Kubernetes

Architecture, intuitions, real-world use cases, and the edge cases that separate operators from experts — with diagrams for every core mechanism.

Aligned to **Kubernetes 1.36 “Haru”** (mid-2026) · control plane internals · networking · scheduling · storage · security

A practitioner's reference for building an accurate mental model of how the cluster really works

Contents

1 · How to read this guide

Mastering Kubernetes is not memorizing YAML fields. It's internalizing one loop — *declare desired state, let controllers drive reality toward it* — and then understanding how each subsystem (scheduling, networking, storage, security) participates in that loop.

Almost every Kubernetes behavior, and almost every outage, becomes obvious once you can answer three questions about any object: **What is its desired state? Which controller reconciles it? What is its actual state and why hasn't it converged?** This guide builds that lens, subsystem by subsystem, with a diagram for each core mechanism.

INTUITION
The mental model under the hood — what's really happening, in plain language.

USE CASE
Where it shows up in a real platform, and the pattern to reach for.

EDGE CASE
The non-obvious boundary condition that causes 2 a.m. pages.

PITFALL
A common mistake — and what to do instead.

LEVER
A specific setting or field and how to reason about it.

2 · The one big idea: declarative state & reconciliation

You never tell Kubernetes *how* to do something. You declare *what you want* — "3 replicas of this image, exposed on port 80" — and store that desired state in the cluster. Independent control loops called **controllers** continuously observe actual state, compare it to desired state, and take corrective action to close the gap. This never stops; it's a steady-state loop, not a one-time command.

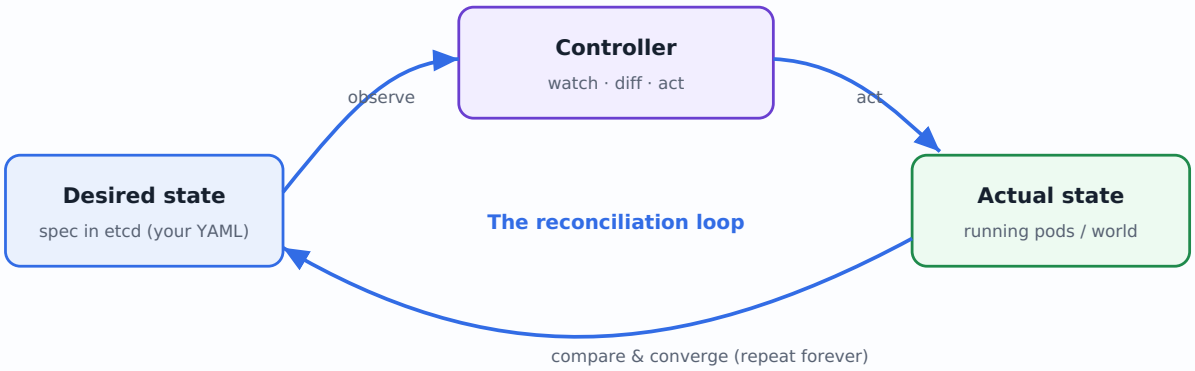

Figure 1 — Every Kubernetes object is governed by a controller running this loop continuously. "Self-healing" is just this loop noticing a gap and closing it.

INTUITION

A pod crashes, a node dies, someone deletes a resource — the loop simply notices "actual ≠ desired" and acts. There is no special "recovery" code path; healing is the *same* reconciliation that created things in the first place. Once this clicks, Kubernetes stops feeling magical and starts feeling predictable.

EDGE CASE

Reconciliation is *level-triggered*, not edge-triggered: controllers act on the current observed state, not on a stream of events. A missed event isn't fatal — the next sync corrects it. This is why Kubernetes is robust, but also why "I deleted it and it came back" happens: the controller's desired state still says it should exist.

3 · Cluster architecture

A cluster splits into the **control plane** (the brain — makes global decisions) and **worker nodes** (the muscle — run your containers). The **API server** is the single front door: every component, every `kubectl`, talks only to it. Components never talk directly to each other — they read and write state through the API server.

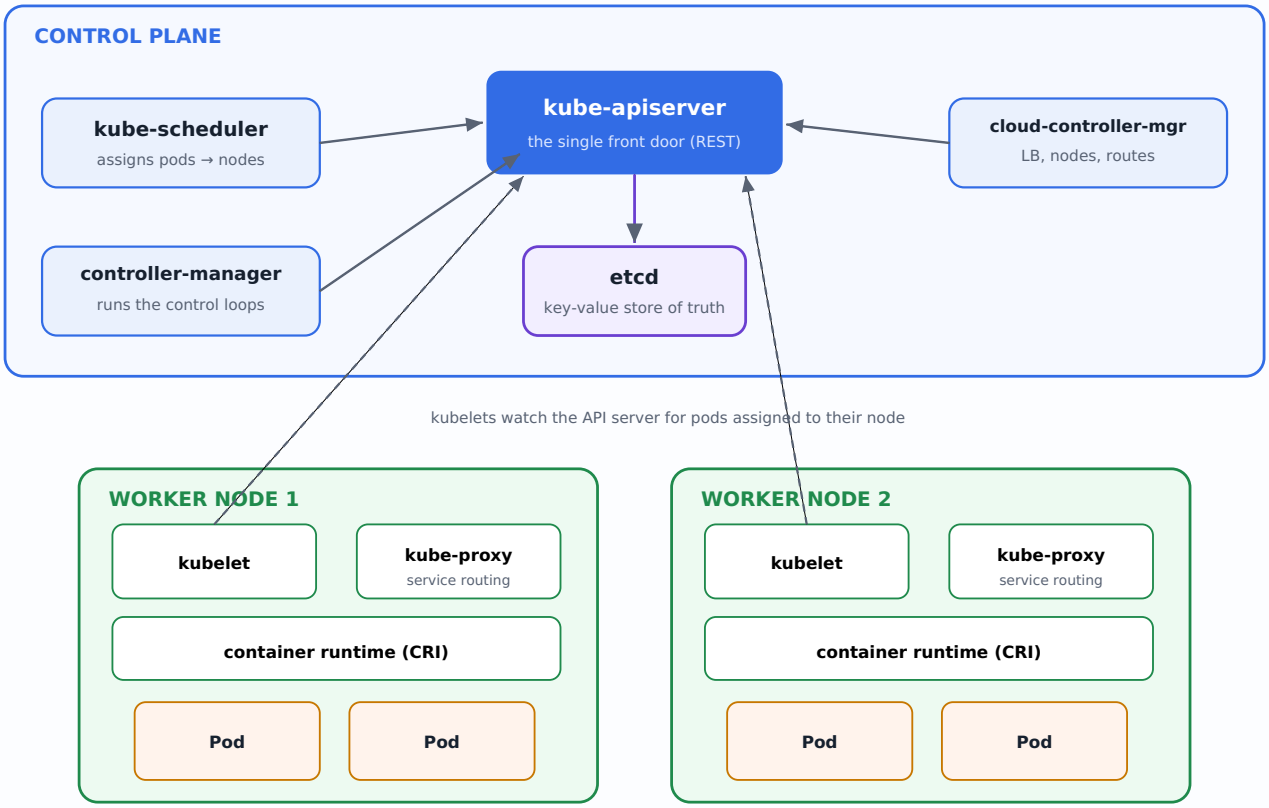

Figure 2 — Hub-and-spoke architecture. The API server is the only component with direct access to etcd; everything else is a client of the API server. This is why the API server's availability defines the cluster's.

Component	Role
kube-apiserver	Validates and serves the API; the only writer to etcd; the hub all traffic flows through.
etcd	Consistent, distributed key-value store holding all cluster state. The source of truth.
kube-scheduler	Watches for unscheduled pods and binds each to the best node.
controller-manager	Bundles the built-in controllers (Deployment, ReplicaSet, Node, Job, …) into one process.
cloud-controller-manager	Integrates with the cloud provider: provisions load balancers, manages node lifecycle and routes.
kubelet	Node agent: ensures the containers described by its pods are running and healthy via the CRI.
kube-proxy	Programs node networking rules so Service virtual IPs route to backing pods.

EDGE CASE

If **etcd** is unhealthy or the **API server** is down, the cluster can't accept changes — but *already-running pods keep running*, because the kubelet operates from its last known state. Control plane down ≠ workloads down. This decoupling is a feature, and why you should never panic-delete pods during a control-plane incident.

PITFALL

etcd is the crown jewels. Lose it without a backup and you've lost the cluster's entire state. Back up etcd regularly, run it on fast low-latency disks, and keep an odd number of members (3 or 5) for quorum — etcd needs a majority to accept writes.

4 · The API request lifecycle

Understanding exactly what happens between `kubectl apply` and a running container demystifies most "why isn't my pod starting?" debugging.

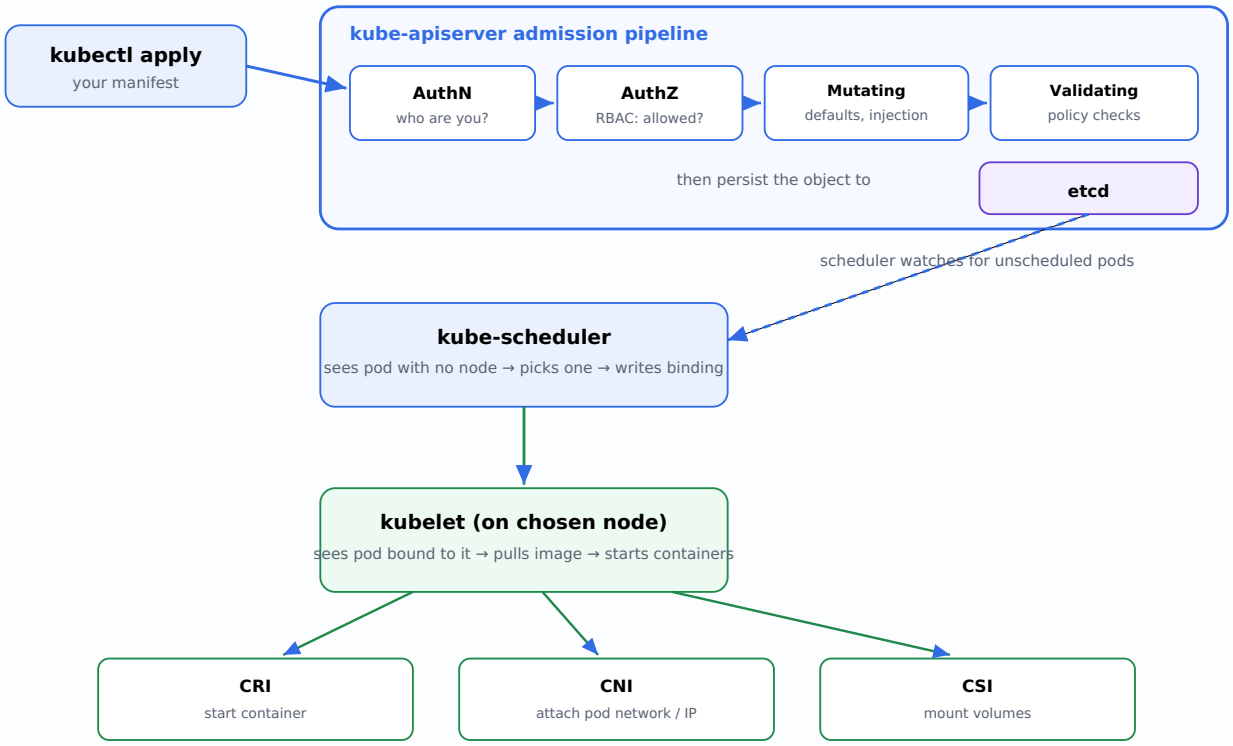

Figure 3 — Object lifecycle. Notice that the scheduler and kubelet never talk to each other or to kubectl; they each independently watch the API server and react. Every arrow into the control plane is a write to etcd that someone else later observes.

INTUITION

Creation is a relay of *independent watchers*, not a call stack. The API server just records facts ("this pod exists, unscheduled"); the scheduler later notices and records another fact ("it's bound to node X"); the kubelet on X later notices and acts. Decoupling via the shared API store is what makes the system resilient and horizontally scalable.

USE CASE — ADMISSION WEBHOOKS

The mutating/validating stages are your policy enforcement point. Service meshes inject sidecars here; policy engines reject pods that run as root or lack resource limits here. In 1.36, CEL-based **admission policies** are GA — in-process, no webhook server to operate.

EDGE CASE

A pod stuck in `Pending` means it cleared admission and was persisted, but the *scheduler* couldn't place it (no node satisfies its resources/affinity/taints) — or it's bound but the *kubelet* can't start it (image pull error, volume won't mount). `kubectl describe pod` tells you exactly which stage stalled.

5 · The object model

Every resource is an object with a consistent shape: `apiVersion`, `kind`, `metadata` (name, namespace, **labels**, annotations), `spec` (your desired state), and `status` (the controller's report of actual state). You write `spec`; the system writes `status`.

INTUITION — LABELS & SELECTORS ARE THE GLUE

Kubernetes objects don't reference each other by hard pointers; they're loosely coupled by **labels**. A Service finds its pods with a label *selector*, not a list of pod names. This is why you can replace every pod under a Service and it keeps working — the selector matches whatever currently carries the label. Labels are the cluster's join keys.

Concept	What it does
Labels + selectors	Loose coupling: "all pods with `app=web`". The basis of Services, Deployments, affinity, NetworkPolicy.
Annotations	Non-identifying metadata for tools/humans (not selectable).
Namespaces	A scope for names + a boundary for quotas, RBAC, and network policy. Not a security sandbox by themselves.
Owner references	Parent/child links enabling cascading deletion (delete a Deployment → its ReplicaSets and pods are garbage-collected).

EDGE CASE

Namespaces isolate *names and policy*, not the *network*. By default, a pod in namespace `a` can reach a pod in namespace `b` — the flat network spans namespaces. Real isolation requires **NetworkPolicy** (§9). Treating namespaces as a security boundary without network policy is a classic misconception.

6 · Pods — the atom of scheduling

A **Pod** is one or more containers that are always co-located and co-scheduled, sharing a network namespace (one IP, shared `localhost`) and able to share volumes. The pod — not the container — is the smallest unit Kubernetes schedules.

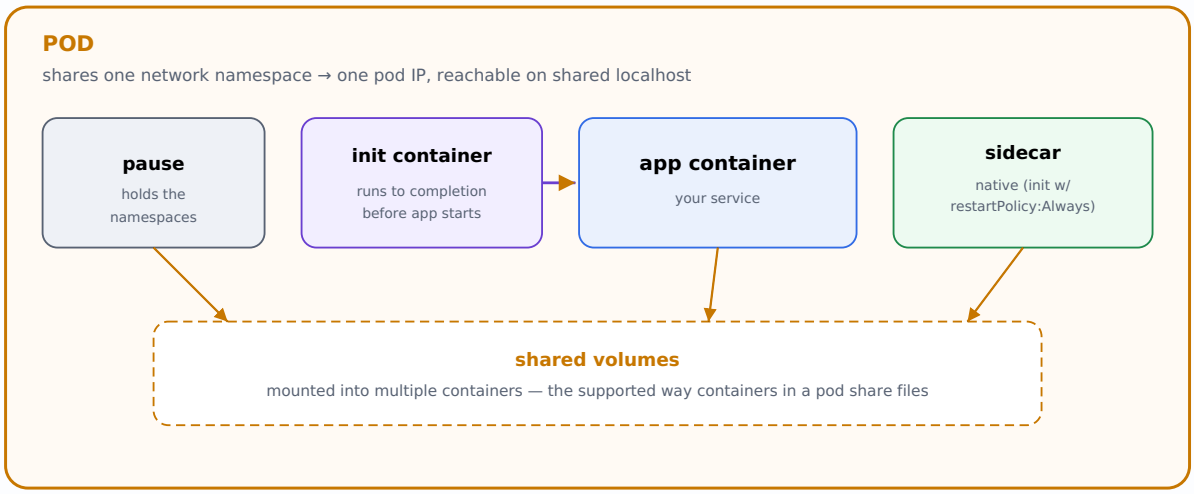

Figure 4 — Pod internals. The `pause` container quietly holds the shared namespaces so app containers can come and go without losing the pod's IP. Native sidecars (GA) are init containers that keep running, with guaranteed start-before / stop-after ordering.

HEALTH PROBES — HOW THE KUBELET KNOWS WHAT "HEALTHY" MEANS

Probe	Question	On failure
startup	Has the app finished booting?	Holds off the other probes; kills if it never starts
readiness	Can it serve traffic *right now*?	Removed from Service endpoints (no traffic) — *not* restarted
liveness	Is it alive, or wedged?	Container is *restarted*

PITFALL — THE LIVENESS DEATH SPIRAL

Pointing a liveness probe at a heavy or dependency-coupled endpoint is dangerous: under load the endpoint slows, the probe times out, the kubelet restarts the container, which makes things worse cluster-wide. Liveness should check "is this process wedged?", cheaply. Use *readiness* to shed traffic during slowness; reserve *liveness* for true deadlocks. And always set a `startup` probe for slow-booting apps so liveness doesn't kill them mid-boot.

EDGE CASE

Pods are *mortal and ephemeral*. When a node dies, its pods are gone forever — they are not migrated; new pods are created elsewhere with new names and new IPs. Never store durable data in a pod's writable layer, and never hardcode a pod IP. This impermanence is exactly why Services, controllers, and PersistentVolumes exist.

7 · Controllers & workloads

You rarely create bare pods. You declare a higher-level workload object, and its controller manages pods for you — recreating, scaling, and updating them.

Workload	Use it for	Key trait
Deployment	Stateless apps (web, APIs)	Manages ReplicaSets; rolling updates & rollbacks
StatefulSet	Databases, anything with stable identity	Stable network names + per-pod persistent storage; ordered ops
DaemonSet	Node agents (logging, CNI, monitoring)	Exactly one pod per (matching) node
Job	Run-to-completion batch work	Tracks successful completions; retries failures
CronJob	Scheduled jobs	Creates Jobs on a cron schedule

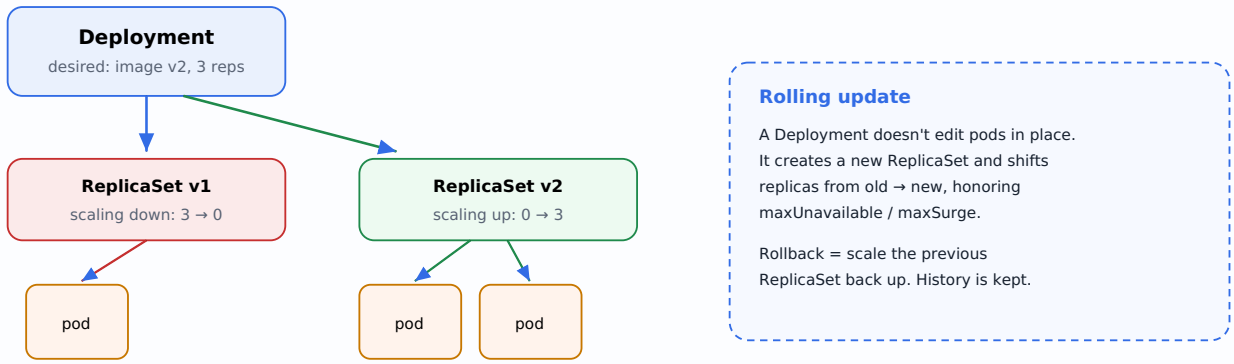

Figure 5 — A Deployment manages ReplicaSets, and ReplicaSets manage pods. Updates roll forward by shifting replicas between versioned ReplicaSets — which is exactly what makes instant rollback possible.

EDGE CASE — STATEFULSET IDENTITY

StatefulSet pods get *stable, ordinal* names (`db-0` , `db-1`) and each keeps its *own* PersistentVolume across rescheduling. Scaling and rolling updates happen in order. This is essential for clustered databases that care which member is which — but it also means operations are slower and deletion doesn't remove the PVCs by default (your data is protected, but you must clean up storage deliberately).

PITFALL

Setting `replicas` in your manifest while an HPA also controls the same Deployment creates a tug-of-war: every `apply` resets the count and fights the autoscaler. When an HPA owns a workload, omit `replicas` from the manifest (or use a server-side-apply field manager that yields it).

8 · Scheduling

When a pod needs a home, the scheduler runs a two-phase pipeline over all nodes: **filtering** (which nodes *can* run it?) then **scoring** (which is *best*?). It binds the pod to the winner.

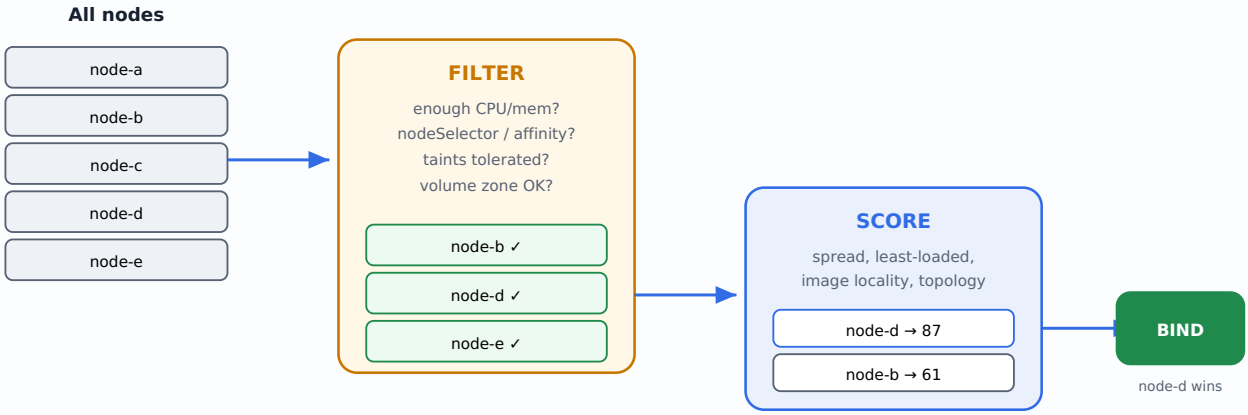

Figure 6 — The scheduler funnels every node through hard constraints (filter) then soft preferences (score). If filtering eliminates all nodes, the pod stays `Pending`.

THE CONTROLS YOU PLACE ON SCHEDULING

- **nodeSelector / node affinity** — attract pods to nodes with certain labels (e.g. `gpu=true`).
- **Pod affinity / anti-affinity** — co-locate pods, or keep them apart (e.g. spread replicas across nodes).
- **Taints & tolerations** — the inverse: a node *repels* pods unless they explicitly tolerate the taint (e.g. dedicated GPU nodes).
- **Topology spread constraints** — evenly distribute pods across zones/nodes for resilience.

INTUITION — AFFINITY VS. TAINTS

Affinity is a pod saying "I *want* to be there." A taint is a node saying "stay *away* unless you have a pass." Use affinity to pull workloads toward suitable hardware; use taints to reserve nodes for specific workloads and keep everything else off. They're complementary, not redundant.

EDGE CASE

The scheduler decides placement *once*, at bind time, using a snapshot. It does not re-evaluate later — if the cluster becomes unbalanced, pods don't move on their own. Rebalancing needs an external tool (a descheduler) or a pod restart. "Why is everything on one node?" is usually this.

9 · Networking — the flat model

Kubernetes mandates one networking rule: **every pod gets its own IP, and every pod can reach every other pod directly, without NAT**. A **CNI** plugin (Calico, Cilium, etc.) implements this flat network. On top of it sit Services, which provide stable addressing over ephemeral pods.

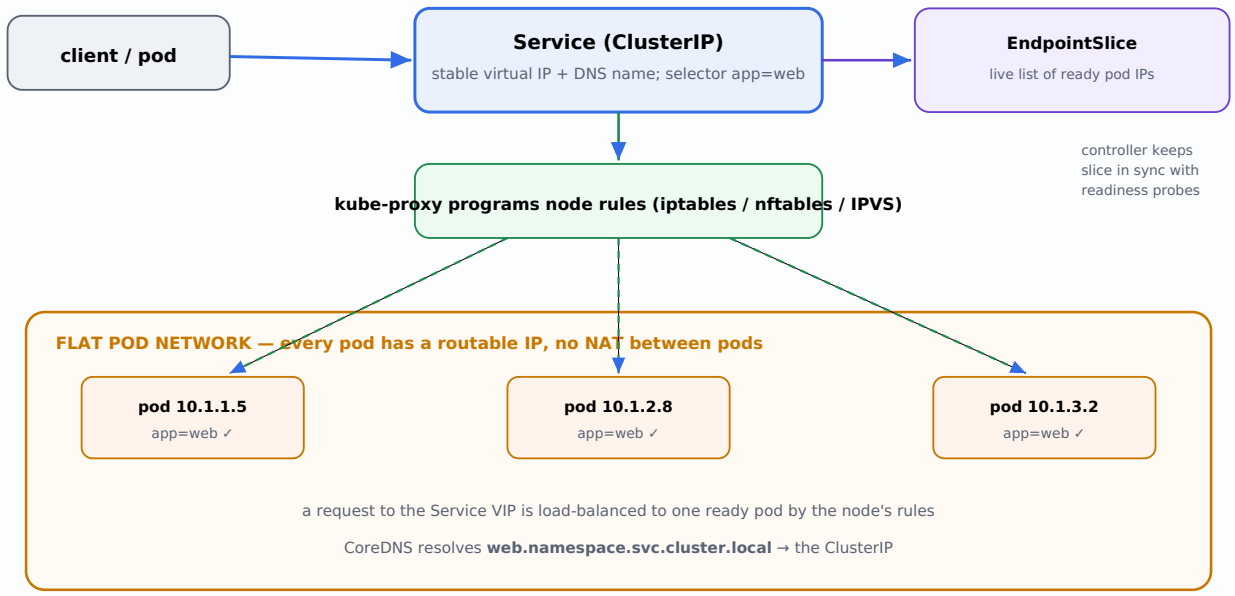

Figure 7 — A Service is a stable virtual IP and DNS name backed by a live, readiness-filtered list of pod IPs (EndpointSlice). kube-proxy turns "send to the VIP" into "send to a real pod." Pods come and go; the Service address never changes.

SERVICE TYPES — LAYERED FROM INSIDE TO OUTSIDE

Type	Reachable from	How
ClusterIP	Inside the cluster only	Virtual IP + DNS (the default)
NodePort	Any node's IP:port	Opens a high port on every node
LoadBalancer	The internet	Provisions a cloud LB (builds on NodePort)
Headless (`clusterIP: None`)	Inside; direct to pods	DNS returns pod IPs, no VIP — used by StatefulSets

INTUITION

A Service is not a process — it's a *rule*. There's no proxy pod sitting in the path; kube-proxy installs kernel routing/NAT rules on every node so the virtual IP transparently rewrites to a real pod IP. That's why Services have no CPU cost and no single point of failure.

EDGE CASE — DNS & READINESS COUPLING

A pod only joins a Service's endpoints once its *readiness* probe passes. Misconfigure readiness and a healthy pod silently receives no traffic, or a broken pod keeps receiving it. Most "service returns errors intermittently" incidents trace back to readiness, not the network.

USE CASE — NETWORKPOLICY

By default all pods can talk to all pods. To enforce zero-trust segmentation (e.g. only the API tier may reach the database), apply a **NetworkPolicy**. Note it's enforced by the CNI — a CNI that doesn't support policy will silently ignore it, a dangerous false sense of security.

10 · Ingress vs. Gateway API — L7 routing

Services give you L4 (IP/port). To route HTTP by host and path, terminate TLS, and do header-based routing, you need an L7 layer. Historically that was **Ingress**; the modern, more expressive successor is the **Gateway API**.

	Ingress	Gateway API
Model	One object, controller-specific annotations	Role-oriented: Gateway (infra) + HTTPRoute (app)
Expressiveness	Basic host/path	Header/method/weight routing, traffic splitting, cross-namespace
Status	Stable but feature-frozen	The strategic direction in 2026

EDGE CASE — INGRESS-NGINX RETIREMENT

The widely used **Ingress-NGINX** controller was *retired* by the project in March 2026 (no further fixes or security patches). Existing installs keep running, but new platforms should standardize on the **Gateway API** with a maintained implementation. If you still depend on Ingress-NGINX, plan a migration — running an unmaintained edge proxy is a live security risk.

11 · Storage — decoupling pods from disks

Because pods are ephemeral, durable data lives in **PersistentVolumes (PV)**. A pod requests storage abstractly via a **PersistentVolumeClaim (PVC)**; a **StorageClass** dynamically provisions a real backing disk through a **CSI** driver. The pod never names a physical disk — it asks for "10Gi, fast" and the system fulfills it.

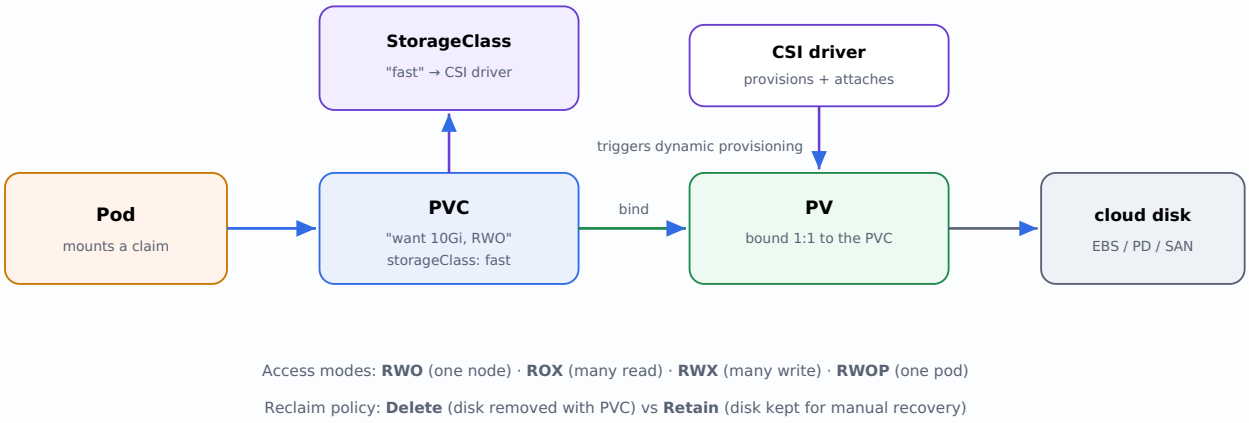

Figure 8 — The PVC is the contract between app teams ("I need storage") and platform teams ("here's how storage is provisioned"). Dynamic provisioning means no human pre-creates disks.

EDGE CASE — RWO AND RESCHEDULING

The most common access mode, **ReadWriteOnce**, allows the volume on *one node* at a time. If a pod must move to another node, the volume has to detach from the old node first — and if the old node is unreachable (not cleanly dead), the attach can hang, leaving the new pod stuck in `ContainerCreating`. This is why single-writer stateful workloads need careful failover handling.

PITFALL

A StorageClass with `reclaimPolicy: Delete` means deleting the PVC *destroys the underlying disk and its data*. For anything precious, use `Retain`, and back up at the storage layer — Kubernetes will happily delete data on your instruction, with no undo.

12 · Configuration & Secrets

Keep configuration out of images. **ConfigMaps** hold non-sensitive config; **Secrets** hold sensitive data. Both are injected as environment variables or mounted files.

PITFALL — SECRETS AREN'T ENCRYPTED BY DEFAULT

A Secret is only **base64-encoded**, not encrypted — anyone who can read the object or etcd sees the value. Enable **encryption at rest** for Secrets (or use an external KMS / secrets store), and lock down RBAC so few principals can read them. Treating base64 as security is a serious mistake.

EDGE CASE

Config injected as *environment variables* is captured at container start and never updates — changing the ConfigMap requires a pod restart. Config injected as a *mounted volume* updates in place (eventually), but your app must re-read the file to notice. Pick the mechanism that matches whether you need live reloads.

13 · Resources, limits & QoS

Each container can declare **requests** (guaranteed minimum, used for scheduling) and **limits** (hard ceiling). The relationship between them determines the pod's **Quality of Service** class, which decides who gets evicted first under pressure.

QoS class	Condition	Eviction order
Guaranteed	requests == limits for all resources	Last to be evicted
Burstable	requests < limits (or only some set)	Middle
BestEffort	no requests or limits	First to be evicted

INTUITION — CPU AND MEMORY BEHAVE DIFFERENTLY

This asymmetry trips up everyone: exceeding a **CPU** limit gets you *throttled* (slowed, but alive). Exceeding a **memory** limit gets you *OOMKilled* (terminated) — memory is incompressible, you can't "slow down" RAM usage. So a too-low memory limit causes crashes; a too-low CPU limit causes mysterious latency.

PITFALL

Omitting **requests** means the scheduler assumes the pod needs ~nothing and packs nodes until they're oversubscribed and start evicting. Omitting **memory limits** lets one leaky pod consume a node and take neighbors down. Always set memory requests *and* limits; set CPU requests, and be cautious with CPU limits (aggressive limits cause needless throttling).

EDGE CASE — IN-PLACE RESIZE

As of recent releases, **in-place pod resize** is GA: you can change CPU/memory requests and limits *without recreating the pod*, avoiding a restart for right-sizing. Memory *decreases* may still require care, but this removes a long-standing pain point for stateful and slow-starting workloads.

14 · Autoscaling

Three autoscalers operate at different layers and compose: **HPA** (more pods), **VPA** (bigger pods), and the **Cluster Autoscaler** (more nodes).

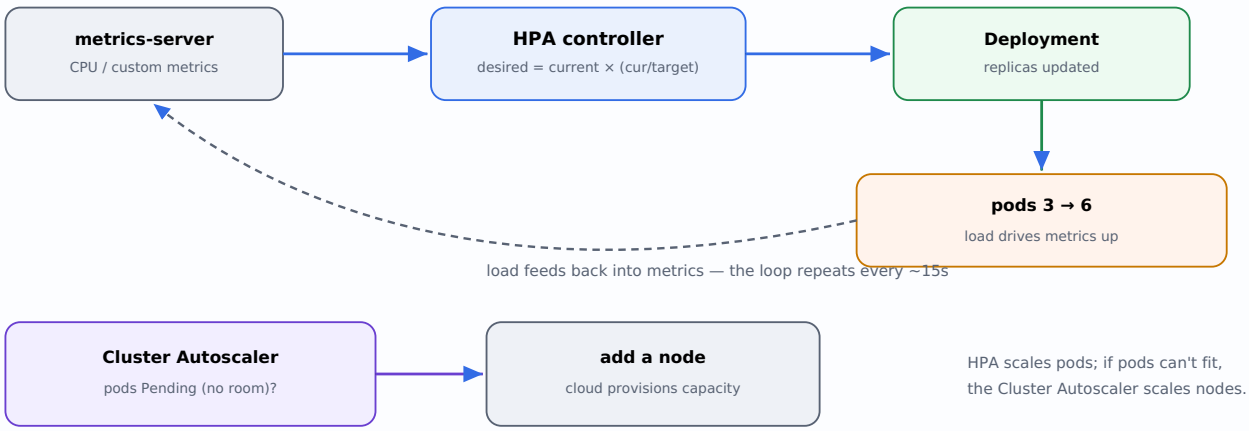

Figure 9 — HPA and Cluster Autoscaler form a two-level loop: HPA adds pods to meet demand; when pods go `Pending` for lack of capacity, the Cluster Autoscaler adds nodes. They're independent controllers that compose through pod state.

EDGE CASE

HPA can only scale on metrics it can *see*. Out of the box that's CPU/memory via metrics-server; scaling on queue depth or requests-per-second needs a custom/external metrics adapter (or **KEDA** for event-driven scaling). And HPA scaling is bounded by your `requests`: if requests are set wrong, the utilization math is wrong and scaling misbehaves.

PITFALL

Don't run HPA and VPA on the *same* resource (both targeting CPU) — they fight. Common safe combo: VPA right-sizes memory requests while HPA scales replica count on CPU/custom metrics.

15 · Security — defense in depth

Kubernetes security is layered: authenticate, authorize, admit, then constrain at runtime.

- **RBAC** — Roles grant verbs on resources; RoleBindings attach them to subjects (users, groups, ServiceAccounts). Grant least privilege; audit who can `get secrets` and who has cluster-admin.
- **ServiceAccounts** — pod identity. Pods authenticate to the API with short-lived, projected tokens. Don't mount the default token if the pod never calls the API.
- **Pod Security Standards** — the built-in *baseline* and *restricted* profiles, enforced per-namespace by the Pod Security admission controller (replacing PodSecurityPolicy).
- **securityContext** — run as non-root, read-only root filesystem, drop Linux capabilities, no privilege escalation.
- **Admission policies** — CEL-based validating/mutating policies (GA in 1.36) enforce org rules in-process, no webhook to run.

INTUITION

Think in terms of *blast radius*. A compromised pod that runs as root, mounts the host filesystem, has a powerful ServiceAccount token, and faces an open network can pivot to the whole cluster. Each control — non-root, minimal RBAC, NetworkPolicy, dropped capabilities — shrinks the radius. Security is the product of these layers, not any single one.

PITFALL

Over-broad RBAC is the quiet killer. A `ClusterRoleBinding` to `cluster-admin` "to make it work" hands the keys to the cluster. Wildcard verbs/resources (`*`) and the ability to create pods or read all secrets are effectively privilege escalation. Review bindings regularly.

16 · Extensibility — making Kubernetes your own

Kubernetes is a platform for building platforms. The same declarative+reconcile model is open to *your* objects.

- **Custom Resource Definitions (CRDs)** — register new object kinds (e.g. `PostgresCluster`) that behave like native ones.
- **Operators** — a CRD plus a custom controller that encodes operational knowledge (provisioning, backups, failover) as a reconciliation loop. The way complex stateful software is run on Kubernetes.
- **Admission webhooks / policies** — inject and enforce at the API boundary.
- **CRI / CNI / CSI** — pluggable runtime, network, and storage so vendors integrate without forking core.

INTUITION — THE OPERATOR PATTERN

An operator is "a human SRE's runbook, written as a controller." Instead of a person watching a database and reacting, a control loop watches a `PostgresCluster` object and continuously reconciles reality to its spec — failover, backups, version upgrades. This is the deepest expression of the one big idea (§2): extend reconciliation to *any* domain.

17 · Observability & debugging

A reliable debugging order, from cheapest to deepest:

1. **`kubectl get`** — what's the high-level status? `Pending`, `CrashLoopBackOff`, `ImagePullBackOff`, `Running` each point somewhere specific.
2. **`kubectl describe`** — read the *Events* at the bottom. This is where the scheduler, kubelet, and controllers narrate *why* something is stuck.
3. **`kubectl logs`** (add `--previous` for a crashed container) — the app's own story.
4. **Metrics & events** — Prometheus for trends; the events stream for recent cluster decisions (note: events are short-lived, typically ~1 hour).

Symptom	Usual meaning
`Pending`	Scheduler can't place it: insufficient resources, unsatisfiable affinity/taints, or unbound PVC
`ImagePullBackOff`	Bad image name/tag or missing registry credentials
`CrashLoopBackOff`	Container starts then exits repeatedly — read logs `--previous`
`OOMKilled`	Exceeded memory limit — raise the limit or fix the leak
`ContainerCreating` (stuck)	Volume won't attach/mount, or CNI can't assign an IP

INTUITION

Almost every debug session is "find the gap and find the controller explaining it." `describe`'s event log is the controller telling you, in its own words, why actual ≠ desired. Reach for it before logs — it usually names the problem outright.

18 · Production edge cases & anti-patterns

Anti-pattern	Why it hurts	Do instead
No resource requests/limits	Oversubscription, evictions, noisy neighbors	Set memory req+limit; CPU req
Liveness probe on a heavy endpoint	Restart storms under load	Cheap liveness; readiness sheds traffic
Treating namespaces as network isolation	Cross-namespace traffic flows freely	Add NetworkPolicy
Secrets unencrypted, broad read access	Plaintext exposure	Encryption at rest + tight RBAC
`:latest` image tags	Non-reproducible, surprise updates	Pin immutable tags / digests
Single replica for "HA" services	Any disruption = outage	≥2 replicas + PodDisruptionBudget + anti-affinity
`cluster-admin` everywhere	No blast-radius containment	Least-privilege RBAC
State in pod's writable layer	Data lost on reschedule	PersistentVolumes
Ignoring PodDisruptionBudgets	Node drains take down all replicas at once	Define PDBs for critical apps
Relying on Ingress-NGINX (retired)	Unmaintained edge = security risk	Migrate to Gateway API

EDGE CASE — PODDISRUPTIONBUDGETS & NODE DRAINS

During upgrades, nodes are *drained* (pods evicted). Without a **PodDisruptionBudget** declaring "keep at least N available," a drain can evict every replica of a service simultaneously, causing an outage *during routine maintenance*. PDBs make voluntary disruptions respect your availability needs — essential for anything that must stay up through upgrades.

19 · A mental model & mastery checklist

THE ONE IDEA TO KEEP

Everything is a control loop closing the gap between *desired* and *actual* state, coordinated through a shared API store. Master that and each subsystem becomes a variation on the same theme: the scheduler reconciles "where should pods run," the Service controller reconciles "which pods back this VIP," an operator reconciles "what does a healthy database look like."

You've internalized Kubernetes when, without looking it up, you can:

- Trace a request from `kubectl apply` through admission, etcd, scheduler, and kubelet (Fig. 3).
- Explain why a pod is `Pending` vs `CrashLoopBackOff` vs `ContainerCreating` — and which component to ask.
- Choose Deployment vs StatefulSet vs DaemonSet vs Job for a workload and justify it.
- Reason about how a Service finds its pods, and why readiness gates traffic (Fig. 7).
- Predict eviction order from QoS class, and know CPU throttles while memory OOMKills.
- Place workloads deliberately with affinity, taints, and topology spread (Fig. 6).
- Protect data with the right access mode and reclaim policy (Fig. 8), and stateful identity with StatefulSets.
- Shrink blast radius with non-root, least-privilege RBAC, NetworkPolicy, and Pod Security.
- Debug by reading the *events* in `describe` before reaching for logs.
- Recognize when to extend the platform with a CRD + operator instead of bolting on scripts.

Built for Kubernetes 1.36 “Haru” (mid-2026). Feature gates, defaults, and managed-platform specifics (EKS / GKE / AKS) vary by version and provider — always confirm against the docs for your exact cluster version before relying on a behavior in production.